

Research Note: Low-Level Design (LLD) Practice Platform

A Focused MVP for Repeatable, Rubric-Based Software Architecture Practice

Evaluator Engine: Google Gemini 3.6 Flash

1. LEARNER PROBLEM & GAPS

Target Learner: Software engineers preparing for Low-Level Design (LLD) and Object-Oriented Design (OOD) interviews, as well as developers transitioning from procedural/algorithmic code to modular, decoupled system architectures.

Why LLD Practice Is Difficult

Unlike algorithmic coding where deterministic test suites give instant pass/fail validation, LLD has no automated compiler for architectural quality. Multiple valid patterns can solve the same problem (e.g. State vs. Strategy). Learners struggle with identifying bloated "God classes", knowing when to favor composition over inheritance, and designing for concurrency race conditions on shared state.

Gaps in Existing Approaches

- **Static Books & Video Courses (e.g., Grokking LLD):** Present single passive reference solutions. They show one author's design, but cannot evaluate a learner's own attempt or diagnose their specific architectural trade-offs.
- **Peer Mock Interviews (e.g., Pramp):** Difficult to schedule repeatedly, expensive, and subject to high interviewer variance and subjective bias.
- **Generic AI Chatbots (ChatGPT / Claude):** When prompted freely, they provide superficial praise, miss missing concurrency locks, and lack a calibrated grading rubric for consistent assessment.

2. PRODUCT DIRECTION & CORE LEARNER LOOP

Product Vision: Build a focused practice platform providing immediate, rubric-based, evidence-grounded evaluation on user-authored designs, enabling deliberate practice and measurable improvement.

Choose Problem → Think / Design → Submit → Get Feedback → Review → Try Again

The platform emphasizes rapid iteration: learners inspect actionable critiques, refactor their entity responsibilities or relationships in the workspace, and resubmit to measure score improvement.

3. MVP SCOPE & SYSTEM BOUNDARIES

Curated Problem Set (4 Scenarios)

- **Parking Lot (Medium):** Strategy pattern for fee calculation; concurrency locks on spot allocation.
- **Elevator System (Hard):** State pattern for cabin states; scheduling algorithms and dispatch queues.
- **Vending Machine (Easy):** State machine transitions and inventory encapsulation.
- **Library System (Medium):** Entity relationships, member rules, and fine calculation.

Structured 6-Section Canvas

- Requirements & Assumptions
- Core Classes & Interfaces
- Class Responsibilities (SRP & Cohesion)
- Relationships (Composition vs. Inheritance)
- Design Decisions & Trade-offs
- Edge Cases & Concurrency

Explicit MVP Non-Goals (Out of Scope)

Large-scale High-Level Design (Kubernetes, sharding, distributed caches, CDN), user authentication / LMS overhead, and compiled code execution sandboxes. The focus is strictly on architectural reasoning and OOP decomposition.

4. EVALUATION APPROACH

A hybrid dual-tier engine balances fast deterministic structural validation with deep semantic reasoning:

- **Deterministic Checks (RuleBasedEvaluator):** Validates section completeness before invoking AI, checks for class/interface declarations and concurrency primitives, and short-circuits empty submissions to conserve resources.
- **Rubric-Based Evaluation (LLMEvaluator via gemini-3.6-flash):** Evaluates architectural quality against a strict JSON schema (responseSchema in @google/genai) across 7 weighted criteria:

Rubric Criterion	Weight	Core Architectural Focus
Requirement Understanding	15%	Functional completeness, domain entity representation, constraint adherence
Class Responsibilities & Cohesion	20%	Single Responsibility Principle (SRP), avoidance of God classes
Encapsulation & Interfaces	15%	Information hiding, interface segregation, access modifiers
Relationships & Coupling	15%	Composition over inheritance, loose coupling, dependency injection
Extensibility & Trade-offs	15%	Open/Closed Principle (OCP), design pattern applicability (Strategy, State)
Edge Cases & Testability	10%	Concurrency, thread safety, race conditions, unit test isolation
Explanation Quality	10%	Coherence, technical precision, and architectural reasoning clarity

Feedback Model (Evidence → Concern → Suggestion): Every critique quotes extracted evidence text directly from the candidate's submission, explains the architectural concern, and provides concrete refactoring guidance.

5. KEY PRODUCT & ENGINEERING DECISIONS

- **Structured Canvas over Free-form Text:** Enforces the systematic 6-step decomposition expected in technical interviews and prevents incomplete or unstructured submissions.
- **Server-Side API Key Isolation:** All Gemini API calls are executed strictly inside Node.js Vite server middleware (/api/*). GEMINI_API_KEY is never bundled or leaked to client JavaScript.
- **Upfront Persistence Pattern:** Submissions are saved immediately upon clicking submit. If an API timeout or network drop occurs, the candidate's work is preserved in state Failed for 1-click retry.
- **Evaluator Strategy Hierarchy:** PracticeService depends on an Evaluator interface rather than a concrete SDK, enabling sub-second offline unit testing with mocks and future multi-model extensibility.
- **Resilient Recovery:** Employs a 60s timeout boundary and automatic exponential backoff retries for transient Google API HTTP 503 / 429 traffic spikes.

6. TRADE-OFFS & LIMITATIONS

- **In-Memory Storage:** Attempt history resets upon server restart. For an MVP prototype, this avoids database setup friction while keeping storage isolated behind a repository interface (PracticeStore).
- **Textual Representation over UML:** Lacks visual class diagramming, but structured text is significantly faster to draft, edit, and evaluate reliably with an LLM.
- **LLM Phrasing Variance:** While mathematical rubric weights and scores remain stable, qualitative phrasing can vary slightly across runs of the same design.
- **Prioritizing Feedback Depth over Platform Scale:** The prototype focuses on evaluation explainability, actionable critique, and domain design quality rather than distributed multi-tier deployment.